De La Salle University
Department of Software Technology

CCDSALG Project Specifications
Term 3, AY 2025–2026
Deadline: **Jun 23, 2026 (T)** *before* **0800**

1 Overview

In this project, you will develop a C or Java application to parse, convert, and evaluate arithmetic expressions. Your tasks include:

1. implementing and using your own Stack and Queue structures;
2. converting expressions between a pair of notations (Infix, Postfix, or Prefix);
3. evaluating expressions in your assigned format;
4. analyzing algorithm performance both theoretically and empirically; and
5. writing a report.

Notation pairs for conversion and notation for evaluation will be assigned.

2 Application Requirements

2.1 Tokens

Your application must be capable of processing expressions containing multi-digit integers, grouping symbols, and a set of arithmetic operators. Below is the token table along with their respective precedence levels and associativity rules, which your algorithm must strictly follow.

Operator	Description	Precedence Level	Associativity
`( , )`	Parentheses (Grouping)	4 (Highest)	Non-associative
`^`	Exponentiation	3	Right-to-Left
`*`, `/`, `%`	Multiplication, Division, Modulo	2	Left-to-Right
`+`, `-`	Addition, Subtraction	1 (Lowest)	Left-to-Right

2.2 Error Handling

Your program must handle and explicitly report the following:

- Mismatched parentheses e.g., `(3 + 12`.
- Invalid characters/operators e.g., `23 $ 1`.
- Malformed expressions e.g., `A + * B`.
- Division by zero.

3 Implementation

Apply good programming practices by writing well-structured, modular code with the use of clear and descriptive variable names.

The following rules will be strictly enforced. Each violation will lead to a deduction from your project score.

- declaration and use of global variables is not allowed;
- **break** statements may only be found inside **switch** statements;
- **return** statements may only be found in functions where the return type is not **void**;
- **return** statements may not be used to prematurely terminate a loop, a function, or a program;
- the use of a **continue** statement is not allowed;

- the use of exit() statement is not allowed;
- the use of **goto** labels is not allowed; and
- calling the main() function is not allowed.

4 Running Time Analysis

The final report must include an Empirical Runtime vs. Token Count (N) plot. Students should overlay a theoretical linear reference line alongside their actual data markers.

4.1 Theoretical running time analysis

Estimate the running time function of your algorithm. Express the function using asymptotic notations. Discuss also the memory usage/requirement of your algorithm.

4.2 Empirical running time analysis

To validate the efficiency of your custom data structures and your algorithm, you must perform a rigorous empirical running time analysis. The evaluation must measure processing times across the four core test categories, scaled dynamically to analyze asymptotic behavior. Start with token size[1] of $N = 5$, and continue generating expressions until token size $N = 10000$.

You must track, log, and plot execution times specifically across these categories.

1. Evaluate the efficiency of the linear token scanning process and your algorithm implementation under standard operator hierarchy rules (`+`, `-`, `*`, `/`, `%`, `^`).

   e.g. `1 + 2 * 3` ($N = 5$). Expand the expression by chaining sequential terms (e.g., `1 + 2 * 3 / 4 - 5 + 3 * 4 - 5...`) to achieve target token sizes up to $N = 10,000$.

2. Evaluate deeply nested groupings force the operator stack to hold a high volume of elements simultaneously.

   e.g. `((5 + 3) * 2) ^ (1 + 1)` ($N = 15$). Generate deeply nested matching structures or highly repetitive parenthetical clusters to stress-test stack capacity limits.

3. Evaluate the impact of tokenizer string-handling, large whitespace variance, and negative results.

   e.g. `100 / (2 - 7)` ($N = 9$). Inject multi-digit integers (e.g., 5-to-6 digit numbers) separated by highly irregular space distributions.

4. Analyze how quickly the system catches structural violations and terminates.

   e.g. `5 + * 3` ($N = 4$) or `(4 + 2]` ($N = 5$). Insert a structural error at varying positions of an expression (at the beginning, middle, and very end of an N-token string).

5 Deliverables

The following must be submitted via **AnimoSpace** on or before **Jun 23, 2026 (T), 0800**.

1. Poster

   Upload a one-page A4 academic poster detailing your system's architecture and findings. The poster must clearly present the following:

   (a) **Algorithm Design.** An overview of how the algorithm works, and how Stacks and Queues are used in the solution.

   (b) **Test Data Generation.** A summary of your dataset generation method/algorithm for scaling tokens.

   (c) **Runtime Analysis and Discussion.** Performance graphs plotting execution time against token count (N) efficiency. Provide also a brief narrative discussing the results, particularly highlighting stack depth profiles during heavy nesting and any performance observed.

   (d) **Conclusion.** A summary alongside key technical takeaways.

   (e) **References.** List of references for algorithms and literature used in APA format.

2. Source Code and Datasets

   Upload the following:

   (a) Source code files (`.h`, `.c`, or `.java`)

   (b) Dataset files (`.txt`)

[1] Token counting varies. It is based on your tokenizer design; for instance, the number 123 may be interpreted as single word-level token ($N = 1$) or as character-level tokens ($N = 3$).

(c) Zip file containing source codes, datasets, etc. and a README file with the exact terminal commands required to compile and run your application. (`.zip`)

3. Declarations

   Upload a PDF file with the following.

   (a) AI Disclosure

   State whether AI tools were used as a *learning partner* during this project. Directly copying AI-generated code is strictly forbidden. If AI was used, you must document the specific tools, the extent of their use, your exact prompts, and a breakdown of what you modified versus what was used as-is. Links or transcripts of the full conversation threads must be attached in the Appendix at the end of this document.

   (b) Contribution Declaration

   List the individual contributions of each group member to the project.

6 Rubric for Grading

Table 1: Rubrics for CCDSALG MCO1, Term 3, AY 2025-2026

CRITERIA	COMPLETE	INCOMPLETE	NO MARKS
Stack (15 points)	Correctly implemented the stack data structure. 15 points	Implementation is not entirely correct. 8 points	Not implemented. 0 point
Queue (15 points)	Correctly implemented the queue data structure. 15 points	Implementation is not entirely correct. 8 points	Not implemented. 0 point
Conversion (20 point)	Correctly implemented the conversion algorithm. Processed all operators and grouping symbols correctly. 20 point	Implementation is not entirely correct. Some operators and grouping symbols were not processed correctly. 10 point	Not implemented. 0 point
Evaluation (15 points)	Correctly implemented the evaluation algorithm. Processed all operators correctly. 15 points	Implementation is not entirely correct. Did not process all operators correctly. 8 points	Not implemented. 0 points
Dataset Generation (10 point)	Generates all required test variations involving all operators, large numbers, nested groupings, positional structural errors 10 point	Data generated is not complete. 5 points	Data is generated did not follow the specifications. 0 points
Poster (15 points)	Required details were covered and clearly presented. Appropriate visualizations are used. 15 points	Some required details were not discussed. 8 points	No discussion or documentation. 0 points
Demo and Q&A (10 point)	Group members can confidently answer all questions during the demo. 10 point	Group members can answer all questions during the demo, with some unconvincing or illogical responses. 5 points	The group failed to answer most questions during the demo. 0 point